Support vector machines

Fraida Fund

Contents

In this lecture . . . 2
Maximal margin classifier . . . 3
Binary classification problem . . . 3
Linear separability . . . 3
Separating hyperplane: sign of the score . . . 3
Separating hyperplane: compact form . . . 3
Using the hyperplane to classify . . . 3
Which separating hyperplane is best? . . . 4
Margin . . . 4
Classifier that maximizes the margin . . . 5
Support vectors . . . 5
Constructing the maximal margin classifier . . . 5
Constructing the maximal margin classifier: optimization problem . . . 6
Constructing the maximal margin classifier: what the constraints ensure . . . 7
Problems with MM classifier: non-separable data . . . 7
Problems with MM classifier: not robust . . . 7
Support vector classifier . . . 8
Basic idea . . . 8
Constructing the support vector classifier . . . 8
Support vector . . . 9
Illustration of effect of K . . . 9
K controls bias-variance tradeoff . . . 9
Solution . . . 10
Problem formulation - original . . . 10
Problem formulation - equivalent: rescaled margin . . . 10
Problem formulation - equivalent: budget in objective . . . 10
Background: constrained optimization . . . 11
Background: Illustration . . . 11
Background: Solving with Lagrangian: the Lagrangian . . . 11
Background: Solving with Lagrangian: joint optimization . . . 11
Background: Solving with Lagrangian: constraint not active . . . 12
Background: Solving with Lagrangian: constraint active . . . 12
Background: Active/inactive constraint . . . 12
Background: Primal and dual formulation . . . 13
Problem formulation - Lagrangian primal . . . 13
Problem formulation - Lagrangian dual . . . 13
Partial derivative with respect to $\mathbf{w}$: derivation . . . 14
Partial derivative with respect to $\mathbf{w}$: result . . . 14
Partial derivative with respect to ϵ_i: derivation . . . 15
Partial derivative with respect to ϵ_i: bound on α_i . . . 15
Substituting into the Lagrangian . . . 15

Coefficients of the line 16
Why solve dual problem? 16
Loss function 17
Loss function: hinge loss 17
Loss function: from the optimization problem 17
Compared to logistic regression 17
SVM as weighted sum over samples 18
Relationship between SVM and other models 18
Correlation interpretation: score as inner products 18
Correlation interpretation: final prediction 18

Math prerequisites for this lecture: Constrained optimization (Appendix C in in Boyd and Vandenberghe).

In this lecture

- Maximal margin classifier
- Support vector classifier
- Solving constrained optimization to find coefficients

Maximal margin classifier

Binary classification problem

- N training samples, each with p features $\mathbf{x}_1, \ldots, \mathbf{x}_N \in \mathbb{R}^p$
- Class labels $y_1, \ldots, y_N \in \{-1, 1\}$

Linear separability

The problem is **perfectly linearly separable** if there exists a **separating hyperplane** H_i such that

- all $\mathbf{x} \in C_i$ lie on its positive side, and
- all $\mathbf{x} \in C_j, j \neq i$ lie on its negative side.

In the binary classification case: The data are linearly separable if we can find a hyperplane that places all $y = 1$ points on one side and all $y = -1$ on the other.

Separating hyperplane: sign of the score

The separating hyperplane has the property that for all $i = 1, \ldots, N$,

$$w_0 + \sum_{j=1}^{p} w_j x_{ij} > 0 \text{ if } y_i = 1$$

$$w_0 + \sum_{j=1}^{p} w_j x_{ij} < 0 \text{ if } y_i = -1$$

Separating hyperplane: compact form

Equivalently:

$$y_i \left(w_0 + \sum_{j=1}^{p} w_j x_{ij} \right) > 0 \tag{1}$$

(we mention this compact form because we will use it in our formulation of the classifier.)

Using the hyperplane to classify

Then, we can classify a new sample $\mathbf{x}$ using the sign of

$$z = w_0 + \sum_{j=1}^{p} w_j x_j$$

The score z is real-valued; the final class is

$$\hat{y} = \text{sign}(z).$$

We can use $|z|$ to measure distance from the hyperplane. Larger $|z|$ means the sample is farther from the hyperplane.

Which separating hyperplane is best?

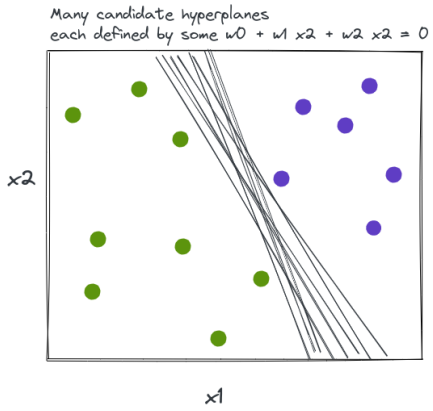


Figure 1: If the data is linearly separable, there are many separating hyperplanes.

We said there will be infinitely many separating hyperplanes, if there is one:

- Previously, with the logistic regression, we found the maximum likelihood classifier: the hyperplane that maximizes the probability of these particular observations.
- This time, we'll find a different one.

Margin

For any "candidate" hyperplane,

- Compute distance from each sample to separating hyperplane.
- Smallest distance among all samples is called the **margin**.

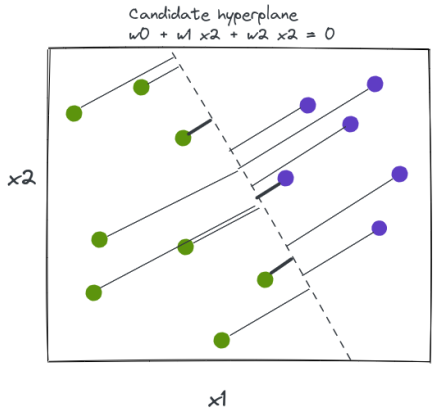


Figure 2: For this hyperplane, bold lines show the smallest distance (tie among several samples).

Classifier that maximizes the margin

- Among all separating hyperplanes, choose the one with the largest margin!
- Find the widest "slab" we can fit between the two classes; use the midline of this "slab".

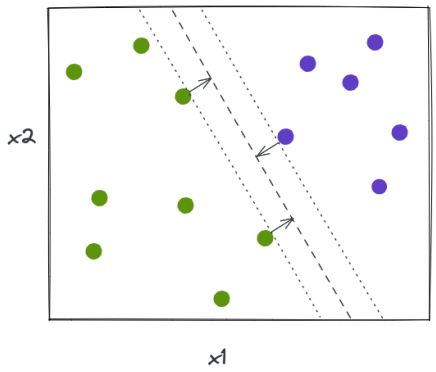


Figure 3: Maximal margin classifier. Width of the "slab" is 2x the margin.

Support vectors

- Points that lie on the border of maximal margin hyperplane are **support vectors**
- They "support" the maximal margin hyperplane: if these points move, then the maximal margin hyperplane moves
- Maximal margin hyperplane is not affected by movement of any other point, as long as it doesn't cross borders!

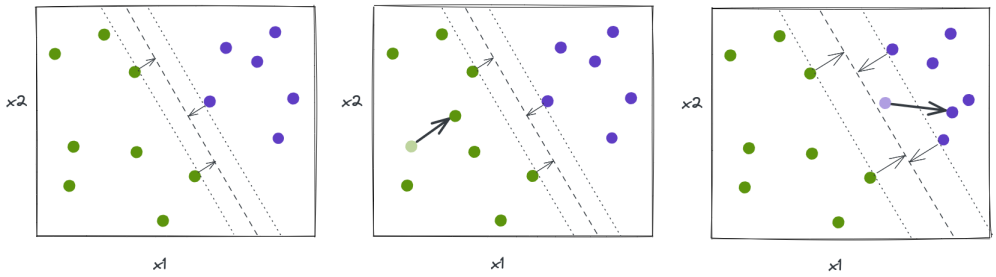


Figure 4: Maximal margin classifier (left) is not affected by movement of a point that is not a support vector (middle) but the hyperplane and/or margin are affected by movement of a support vector (right).

Constructing the maximal margin classifier

To construct this classifier, we will set up a *constrained optimization* problem with:

- an objective
- one or more constraints to satisfy

What should the objective/constraints be in this scenario?

Constructing the maximal margin classifier: optimization problem

$$\underset{\mathbf{w},\gamma}{\text{maximize}}\ \gamma \tag{2}$$

$$\text{subject to: } \sum_{j=1}^{p} w_j^2 = 1 \tag{3}$$

$$\text{and } y_i \left(w_0 + \sum_{j=1}^{p} w_j x_{ij} \right) \geq \gamma, \forall i \tag{4}$$

The constraint

$$y_i \left(w_0 + \sum_{j=1}^{p} w_j x_{ij} \right) \geq \gamma, \forall i$$

guarantees that each observation is on the correct side of the hyperplane *and* on the correct side of the margin, if margin γ is positive. (This is analogous to Equation 1, but we have added a margin.)

The constraint

$$\text{and } \sum_{j=1}^{p} w_j^2 = 1$$

is not really a constraint: if a separating hyperplane is defined by $w_0 + \sum_{j=1}^{p} w_j x_{ij} = 0$, then for any $k \neq 0$, $k \left(w_0 + \sum_{j=1}^{p} w_j x_{ij} \right) = 0$ is also a separating hyperplane.

This "constraint" just scales $\mathbf{w}$ so that distance from ith sample to the hyperplane is given by $y_i \left(w_0 + \sum_{j=1}^{p} w_j x_{ij} \right)$. This is what make the previous constraint meaningful!

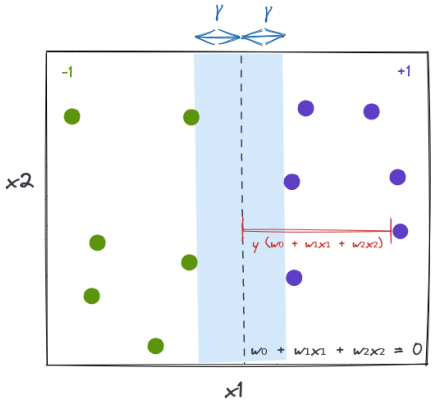


Figure 5: Maximal margin classifier.

Constructing the maximal margin classifier: what the constraints ensure

The constraints ensure that

- Each observation is on the correct side of the hyperplane, and
- at least γ away from the hyperplane

and γ is maximized.

Problems with MM classifier: non-separable data

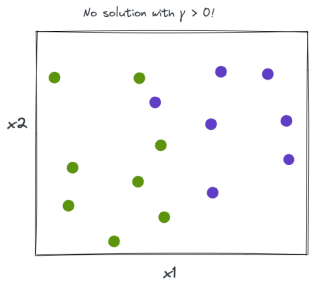


Figure 6: When data is not linearly separable, optimization problem has no solution with $\gamma > 0$.

Problems with MM classifier: not robust

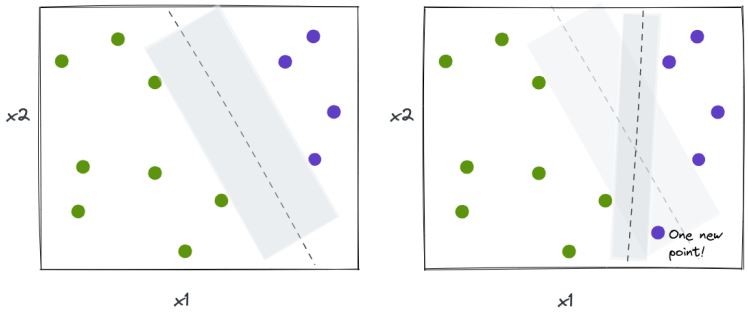


Figure 7: The classifier is not robust - one new observation can dramatically shift the hyperplane.

Support vector classifier

Basic idea

- Generalization of MM classifier to non-separable case
- Use a hyperplane that *almost* separates the data
- "Soft margin"

Constructing the support vector classifier

$$\underset{\mathbf{w},\boldsymbol{\epsilon},\gamma}{\text{maximize}}\ \gamma \tag{5}$$

$$\text{subject to: } \sum_{j=1}^{p} w_j^2 = 1 \tag{6}$$

$$y_i \left(w_0 + \sum_{j=1}^{p} w_j x_{ij} \right) \geq \gamma(1 - \epsilon_i), \quad \forall i \tag{7}$$

$$\epsilon_i \geq 0, \quad \forall i, \quad \sum_{i=1}^{N} \epsilon_i \leq K \tag{8}$$

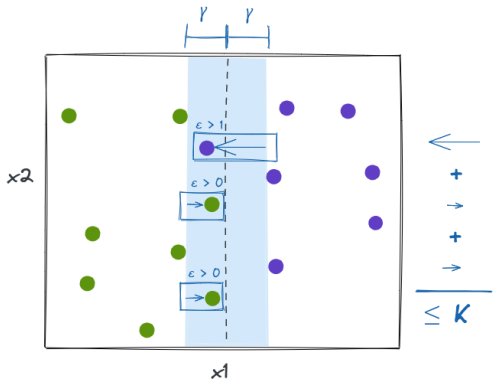


Figure 8: Support vector classifier. Note: the blue arrows show $y_i \gamma \epsilon_i$.

K is a non-negative tuning parameter.

Slack variable ϵ_i determines where a point lies:

- If $\epsilon_i = 0$, point is on the correct side of margin
- If $\epsilon_i > 0$, point has *violated* the margin (wrong side of margin)
- If $\epsilon_i > 1$, point is on wrong side of hyperplane and is misclassified

K is the **budget** that determines the number and severity of margin violations we will tolerate.

- $K = 0 \rightarrow$ same as MM classifier
- $K > 0$, no more than K observations may be on wrong side of hyperplane
- As K increases, more violations allowed, margin can be wider

Support vector

For a support vector classifier, the only points that affect the classifier are:

- Points that lie on the margin boundary
- Points that violate margin

These are the *support vectors*.

Illustration of effect of K

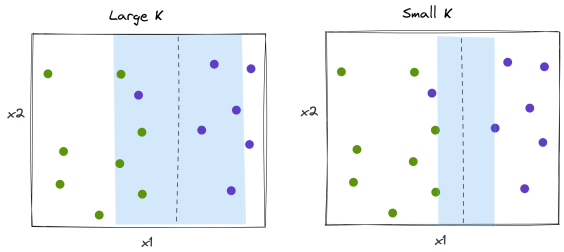


Figure 9: The margin shrinks as K decreases.

K controls bias-variance tradeoff

- Wide margin (K large): many support vectors, low variance, high bias.
- Narrow margin (K small): few support vectors, high variance, low bias.

Terminology note: In ISL and in the first part of these notes, meaning of constant is opposite its meaning in Python `sklearn`:

- ISL and these notes: Large K, wide margin.
- Python `sklearn`: Large C, small margin.

Solution

Problem formulation - original

$$
\begin{aligned}
\underset{\mathbf{w},\boldsymbol{\epsilon},\gamma}{\text{maximize}} \quad & \gamma \\
\text{subject to} \quad & \sum_{j=1}^{p} w_j^2 = 1 \\
& y_i \left(w_0 + \sum_{j=1}^{p} w_j x_{ij} \right) \geq \gamma(1 - \epsilon_i), \quad \forall i \\
& \epsilon_i \geq 0, \quad \forall i \\
& \sum_{i=1}^{N} \epsilon_i \leq K
\end{aligned}
$$

Problem formulation - equivalent: rescaled margin

Remember that scaling $\mathbf{w}$ doesn't change the separating hyperplane. If we scale so that margin boundaries are at $+1$ and -1, then $\gamma = \frac{1}{||w||}$, and we can formulate the equivalent minimization problem:

$$
\begin{aligned}
\underset{\mathbf{w},\boldsymbol{\epsilon}}{\text{minimize}} \quad & \sum_{j=1}^{p} w_j^2 \\
\text{subject to} \quad & y_i \left(w_0 + \sum_{j=1}^{p} w_j x_{ij} \right) \geq 1 - \epsilon_i, \forall i \\
& \epsilon_i \geq 0, \quad \forall i \\
& \sum_{i=1}^{N} \epsilon_i \leq K
\end{aligned}
$$

Problem formulation - equivalent: budget in objective

Next, move the "budget" into the objective function (easier to compute):

$$
\begin{aligned}
\underset{\mathbf{w},\boldsymbol{\epsilon}}{\text{minimize}} \quad & \frac{1}{2} \sum_{j=1}^{p} w_j^2 + C \sum_{i=1}^{N} \epsilon_i \\
\text{subject to} \quad & y_i(w_0 + \sum_{j=1}^{p} w_j x_{ij}) \geq 1 - \epsilon_i, \quad \forall i \\
& \epsilon_i \geq 0, \quad \forall i
\end{aligned}
$$

Background: constrained optimization

Basic formulation of contrained optimization problem:

- **Objective**: Minimize $f(x)$
- **Constraint(s)**: subject to $g(x) \leq 0$

Find x^* that satisfies $g(x^*) \leq 0$ and, for any other x that satisfies $g(x) \leq 0$, $f(x) \geq f(x^*)$.

Background: Illustration

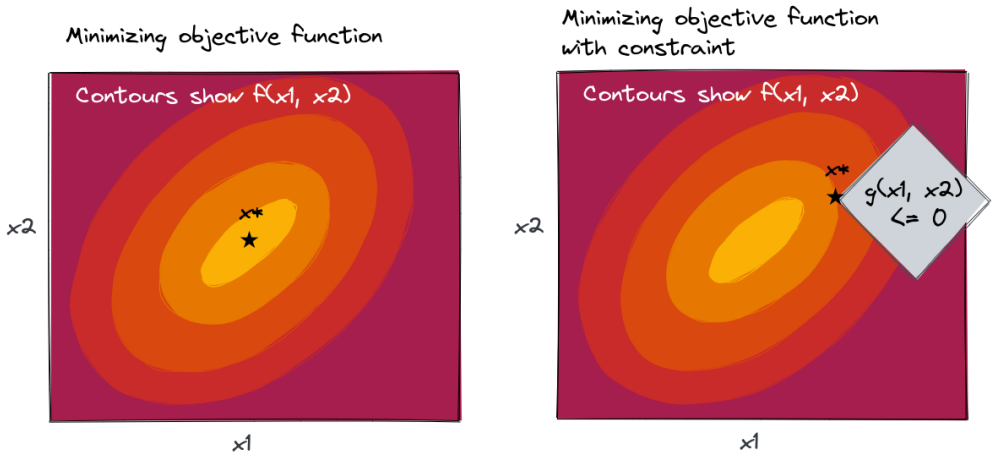


Figure 10: Minimizing objective function, without (left) and with (right) a constraint.

Background: Solving with Lagrangian: the Lagrangian

To solve, we form the Lagrangian:

$$L(x, \lambda) = f(x) + \lambda_1 g_1(x) + \cdots + \lambda_m g_m(x)$$

where each $\lambda \geq 0$ is a *Lagrange multiplier*.

The $\lambda g(x)$ terms "pull" solution toward feasible set, away from non-feasible set.

Background: Solving with Lagrangian: joint optimization

Then, to solve, we use joint optimization over x and λ:

$$\underset{x}{\text{minimize}} \ \underset{\lambda \geq 0}{\text{maximize}} \ f(x) + \lambda g(x)$$

over x and λ.

("Solve" in the usual way for convex function: taking partial derivative of $L(x, \lambda)$ with respect to each argument, and setting to zero. The solution to the original function will be a saddle point in the Lagrangian.)

- We minimize over x (the solution we want)
- We maximize over λ (penalty for violating the constraint)

Background: Solving with Lagrangian: constraint not active

$$\underset{x}{\text{minimize}} \underset{\lambda \geq 0}{\text{maximize}} \; f(x) + \lambda g(x)$$

Suppose that for the x that minimizes $f(x)$, $g(x) \leq 0$ (i.e. x **is in the feasible set.**)

If $g(x) < 0$ (constraint is not active),

- to maximize: we want $\lambda = 0$
- to minimize: we'll minimize $f(x)$, $\lambda g(x) = 0$

Background: Solving with Lagrangian: constraint active

$$\underset{x}{\text{minimize}} \underset{\lambda \geq 0}{\text{maximize}} \; f(x) + \lambda g(x)$$

Suppose that for the x that minimizes $f(x)$, $g(x) > 0$ (x **is not in the feasible set.**)

- to maximize: we want $\lambda > 0$
- to minimize: we want small $g(x)$ and $f(x)$.

In this case, the "pull" between

- the x that minimizes $f(x)$
- and the $\lambda g(x)$ which pulls toward the feasible set,

ends up making the constraint "tight". We will use the x on the edge of the feasible set ($g(x) = 0$, constraint is active) for which $f(x)$ is smallest.

This is called complementary slackness: for every constraint, $\lambda g(x) = 0$, either because $\lambda = 0$ (inactive constraint) or $g(x) = 0$ (active constraint).

Background: Active/inactive constraint

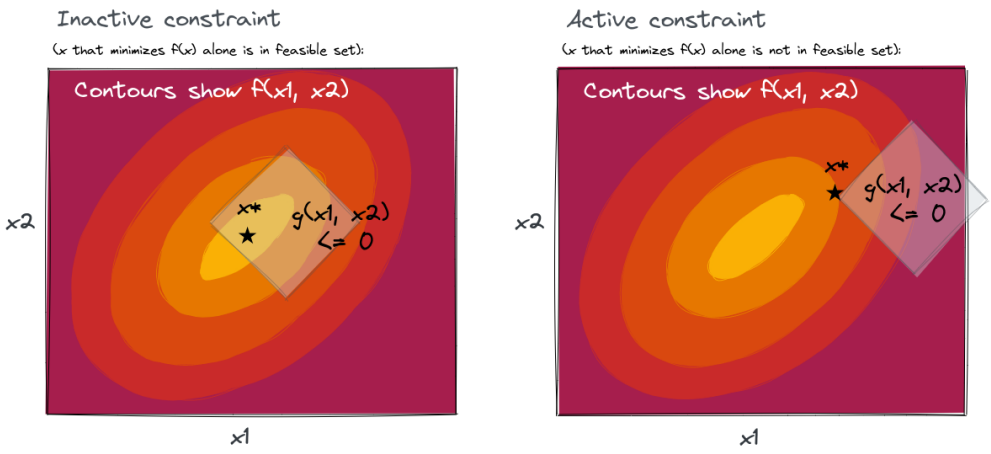


Figure 11: Optimization with inactive, active constraint.

Background: Primal and dual formulation

Under the right conditions, the solution to the *primal* problem:

$$\underset{x}{\text{minimize}}\ \underset{\lambda \geq 0}{\text{maximize}}\ L(x, \lambda)$$

is the same as the solution to the *dual* problem:

$$\underset{\lambda \geq 0}{\text{maximize}}\ \underset{x}{\text{minimize}}\ L(x, \lambda)$$

Problem formulation - Lagrangian primal

Back to our SVC problem - let's form the Lagrangian (introducing α_i multipliers for the contraint on the margin violation and μ_i multipliers for the non-negativity constraint on slack variables):

$$\begin{aligned}\underset{\mathbf{w},\boldsymbol{\epsilon}}{\text{minimize}}\ \underset{\alpha_i \geq 0, \mu_i \geq 0, \forall i}{\text{maximize}}\quad & \frac{1}{2}\sum_{j=1}^{p} w_j^2 \\ & + C\sum_{i=1}^{N} \epsilon_i \\ & - \sum_{i=1}^{N} \alpha_i \left[y_i(w_0 + \sum_{j=1}^{p} w_j x_{ij}) - (1 - \epsilon_i) \right] \\ & - \sum_{i=1}^{N} \mu_i \epsilon_i\end{aligned}$$

This is the *primal* problem.

Problem formulation - Lagrangian dual

The equivalent *dual* problem:

$$\begin{aligned}\underset{\alpha_i \geq 0, \mu_i \geq 0, \forall i}{\text{maximize}}\ \underset{\mathbf{w},\boldsymbol{\epsilon}}{\text{minimize}}\quad & \frac{1}{2}\sum_{j=1}^{p} w_j^2 \\ & + C\sum_{i=1}^{N} \epsilon_i \\ & - \sum_{i=1}^{N} \alpha_i \left[y_i(w_0 + \sum_{j=1}^{p} w_j x_{ij}) - (1 - \epsilon_i) \right] \\ & - \sum_{i=1}^{N} \mu_i \epsilon_i\end{aligned}$$

To solve, we take partial derivatives of the Lagrangian with respect to $\mathbf{w}$ and $\boldsymbol{\epsilon}$, and set them to zero.

Partial derivative with respect to $\mathbf{w}$: derivation

Only two parts of the Lagrangian L depend on $\mathbf{w}$:

$$\frac{1}{2}\sum_{j=1}^{p} w_j^2 \qquad \text{and} \qquad -\sum_{i=1}^{N}\alpha_i y_i \sum_{j=1}^{p} w_j x_{ij}$$

Differentiate:

$$\frac{\partial}{\partial \mathbf{w}}\left(\frac{1}{2}\sum_{j=1}^{p} w_j^2\right) = \mathbf{w}$$

$$\frac{\partial}{\partial \mathbf{w}}\left(-\sum_{i=1}^{N}\alpha_i y_i \sum_{j=1}^{p} w_j x_{ij}\right) = -\sum_{i=1}^{N}\alpha_i y_i \mathbf{x}_i$$

Set derivative to zero:

$$\mathbf{w} - \sum_{i=1}^{N}\alpha_i y_i \mathbf{x}_i = 0$$

Partial derivative with respect to $\mathbf{w}$: result

Therefore:

$$\boxed{\mathbf{w} = \sum_{i=1}^{N}\alpha_i y_i \mathbf{x}_i}$$

where the α_i are the Lagrange multipliers.

Partial derivative with respect to ϵ_i: derivation

Terms involving ϵ_i in the Lagrangian:

$$C\epsilon_i + \alpha_i\epsilon_i - \mu_i\epsilon_i = (C - \alpha_i - \mu_i)\epsilon_i$$

Differentiate:

$$\frac{\partial L}{\partial \epsilon_i} = C - \alpha_i - \mu_i$$

Set equal to zero:

$$C - \alpha_i - \mu_i = 0$$

Therefore:

$$\alpha_i = C - \mu_i$$

Partial derivative with respect to ϵ_i: bound on α_i

Because $\mu_i \geq 0$, we find that α_i must satisfy:

$$\boxed{0 \leq \alpha_i \leq C}$$

Substituting into the Lagrangian

After some algebra, the remaining optimization depends only on α.

$$\begin{aligned} \underset{\alpha_i \geq 0, \forall i}{\text{maximize}} \quad & \sum_{i=1}^{N} \alpha_i - \frac{1}{2} \sum_{i=1}^{N} \sum_{j=1}^{N} \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j \\ \text{subject to} \quad & \sum_{i=1}^{N} \alpha_i y_i = 0 \\ & 0 \leq \alpha_i \leq C, \quad \forall i \end{aligned}$$

The constraint $\sum_i \alpha_i y_i = 0$ comes from setting the partial derivative with respect to w_0 to zero.

This problem is a quadratic program (QP):

- Variables: only the α_i, one per training sample.
- Objective: as shown, it is a concave quadratic in α, so it has a single global maximum.
- Constraints: $\sum_i \alpha_i y_i = 0$ and $0 \leq \alpha_i \leq C$ are linear.

There's no closed-form solution because of the inequality constraints, so a numerical QP solver finds α.

Note that α is non-zero only when the constraint on margin violation is active - only for support vectors.

Coefficients of the line

Once we solve for α, optimal coefficients for $j = 1, \dots, p$ are:

$$\mathbf{w} = \sum_{i=1}^{N} \alpha_i \, y_i \, \mathbf{x}_i$$

and we solve $w_0 = y_i - \sum_{j=1}^{p} w_j x_{ij}$ using any sample i where $0 < \alpha_i < C$ (a support vector exactly on the margin).

Why solve dual problem?

For high-dimension problems (many features), dual problem can be much faster to solve than primal problem:

- Primal problem: optimize over $p + 1$ coefficients.
- Dual problem: optimize over N dual variables, but there are only as many non-zero ones as there are support vectors.

But mainly: the kernel trick, which we'll discuss next, works for the dual formulation, because the data only appears inside inner product $\mathbf{x}_i^T \mathbf{x}_j$!

Loss function

Loss function: hinge loss

This problem is equivalent to minimizing hinge loss:

$$\underset{\mathbf{w}}{\text{minimize}} \left(\frac{1}{2} \sum_{j=1}^{p} w_j^2 + C \sum_{i=1}^{N} \max\left(0,\, 1 - y_i z_i\right) \right)$$

where $z_i = w_0 + \sum_{j=1}^{p} w_j x_{ij}$.

For a labeled observation $(\mathbf{x}_i, y_i)$ with $y_i \in \{-1, 1\}$, let

$$z_i = w_0 + \sum_{j=1}^{p} w_j x_{ij}$$

The hinge loss for this observation is $\max\left(0,\, 1 - y_i z_i\right)$.

Value of $y_i z_i$	Interpretation	Hinge loss: $\max(0,\, 1 - y_i z_i)$
> 1	Correct and outside the margin	0
$= 1$	Right on the margin	0
Between 0 and 1	Correct but inside the margin	$1 - y_i z_i$ (greater than 0)
≤ 0	Misclassified	$1 - y_i z_i$ (greater than 0)

Hinge loss penalizes points only when they are inside the margin or misclassified!

Loss function: from the optimization problem

Each slack variable ϵ_i must satisfy two constraints:

$$\epsilon_i \geq 1 - y_i z_i \qquad \text{and} \qquad \epsilon_i \geq 0$$

The objective penalizes ϵ_i, so at the optimum each ϵ_i is the *smallest* value that satisfies both:

$$\epsilon_i = \max(0,\, 1 - y_i z_i)$$

This is the hinge loss! Substituting it for ϵ_i eliminates the slack variables and constraints, leaving an equivalent unconstrained problem:

$$\underset{\mathbf{w}}{\text{minimize}} \left(\frac{1}{2} \sum_{j=1}^{p} w_j^2 + C \sum_{i=1}^{N} \max\left(0,\, 1 - y_i z_i\right) \right)$$

Compared to logistic regression

- **Hinge loss**: zero for points that are correct and outside margin.
- **Logistic regression loss**: small but non-zero loss for points far from separating hyperplane.

SVM as weighted sum over samples

Relationship between SVM and other models

- Like a logistic regression - linear classifier, separating hyperplane is $w_0 + \sum_{j=1}^{p} w_j x_{ij} = 0$
- Like a weighted KNN - the decision score is a weighted sum of support-vector labels, with weights proportional to the similarity between the test sample and each support vector.

Correlation interpretation: score as inner products

Given a new sample $\mathbf{x}$ to classify, compute

$$\begin{aligned} z(\mathbf{x}) &= w_0 + \sum_{j=1}^{p} w_j x_j \\ &= w_0 + \sum_{i=1}^{N} \alpha_i y_i \left(\sum_{j=1}^{p} x_{ij} x_j \right) \\ &= w_0 + \sum_{i=1}^{N} \alpha_i y_i \left(\mathbf{x}_i^\top \mathbf{x} \right) \end{aligned}$$

Measures inner product (a kind of "correlation") between new sample and each support vector.

Correlation interpretation: final prediction

Final class prediction (assuming $-1, 1$ labels):

$$\hat{y}(\mathbf{x}) = \text{sign}(z(\mathbf{x}))$$

The score $z(\mathbf{x})$ is a weighted sum of support-vector labels; $\hat{y}(\mathbf{x})$ is its sign.